Jackson + RestAssured — JSON Cheat Sheet

Everything you need for parsing JSON responses in RestAssured using Jackson.

JSON Structure Reference

3 tips

Object { }

```
{ "name": "Abhi", "age": "34" }
```

💡 Navigate with .path("key") or .get("key")

Array [ ]

```
[ { "claimNumber": "123" }, { "claimNumber": "124" } ]
```

💡 Loop with: for (JsonNode js : arrayNode)

Leaf value

```
"Abhi"  /  "34"  /  true  /  100
```

💡 End with .asText() / .asInt() / .asBoolean() / .asDouble()

Step 1 — Parse Response

2 tips

RestAssured → JsonNode

```
ObjectMapper mapper = new ObjectMapper();
JsonNode root = mapper.readTree(resp.getBody().asString());
```

💡 Always your starting point

RestAssured → jsonPath (dot notation)

```
resp.jsonPath().getString("data[0].member.name");
resp.jsonPath().getList("data.member.name");
```

💡 Best for quick value extraction

Step 2 — Navigate to Array

3 tips

.path() ✅ Safe (recommended)

```
JsonNode dataNode = root.path("data");
// Returns MissingNode if key not found — no NPE
```

💡 Use when key might be missing

.get() ⚠️ Unsafe

```
JsonNode dataNode = root.get("data");
// Returns null if key not found → NPE risk!
```

💡 Only use when 100% sure key exists

Multiple arrays

```
JsonNode data1Node = root.path("data1");
JsonNode data2Node = root.path("data2");
JsonNode data3Node = root.path("data3");
```

💡 One variable per array key

Step 3 — Loop the Array

2 tips

Basic loop

```
for (JsonNode js : dataNode) {
    // js = one { } object per iteration
}
```

💡 Works for 1 object or 100 objects

❌ Common mistake — .next()

```
// WRONG — grabs only the first object
JsonNode arr = dataNode.elements().next();
for (JsonNode js : arr) { } // iterates fields not objects!
```

💡 Never use .elements().next() to loop

Step 4 — Extract Values

3 tips

Chained .path() — level by level

```
js.path("claimNumber").asText();
js.path("member").path("name").asText();
js.path("member").path("stationary").path("pen").asText();
```

💡 One .path() per { } level

.at() — JSON Pointer with /

```
js.at("/claimNumber").asText();
js.at("/member/name").asText();
js.at("/member/stationary/pen").asText();
```

💡 Always start with / — use / not .

❌ Dot notation does NOT work in Jackson

```
// WRONG — always returns MissingNode
js.path("member.stationary.pen")

// CORRECT
js.path("member").path("stationary").path("pen").asText()
```

💡 Jackson treats the whole string as one key

Step 5 — Convert to Java Type

2 tips

Conversion methods

```
node.asText()       // → String   "Abhi"
node.asInt()        // → int      34
node.asDouble()     // → double   3.14
node.asBoolean()    // → boolean  true
node.toString()     // → JSON string with quotes "Abhi" ❌ avoid
```

💡 Always use asText() for strings — not toString()

Check before extracting

```
node.isMissingNode()  // true if key not found
node.isNull()         // true if value is null
node.isArray()        // true if [ ]
node.isObject()       // true if { }
```

💡 Useful for defensive checks

Full Example

1 tip

Complete flow

```
JsonNode root     = mapper.readTree(resp.getBody().asString());
JsonNode dataNode = root.path("data");

for (JsonNode js : dataNode) {
    String claim  = js.path("claimNumber").asText();
    String name   = js.path("member").path("name").asText();
    String pen    = js.at("/member/stationary/pen").asText();

    System.out.println(claim + " | " + name + " | " + pen);
}
```

💡 The standard template — memorise this

Golden Rules ① `{ }` → `.path("key")` ② `[ ]` → `for loop` ③ leaf → `.asText()` ④ never dot-notation → use `.at("/a/b/c")` ⑤ `.path()` over `.get()` — no NPE